

❄ Slowly Changing Dimensions – Type 1 & Type 2 Implementation (Real-Time Scenario) ❄

Real-Time Business Scenario

Assume we are building a **Retail Data Warehouse**.

Source system: **E-commerce Application**

Customer data arrives daily from the **OLTP system**.

Customer attributes:

- Customer_ID
- Name
- Email
- City
- Phone_Number

Some attributes require **history tracking**, while others do not.

Example rule from business:

Attribute SCD Type

Email Type 1

Phone Type 1

City Type 2

Reason:

- Email corrections do not require history
- Phone corrections do not require history
- City change must maintain history for analytics

Example analysis requirement:

"From which cities did customers place orders in 2023 vs 2024?"

This requires **historical tracking**, so City uses **SCD Type 2**.

Project Architecture

Typical Data Pipeline

Source System (OLTP)



Landing / Raw Layer



Staging Table



SCD Logic (Type1 + Type2)



Customer Dimension Table



Fact Tables / Analytics

Dimension Table Structure

Customer Dimension Table

Column	Description
Customer_SK	Surrogate Key
Customer_ID	Business Key
Name	Customer Name
Email	Email Address
City	Customer City
Phone	Phone Number
Start_Date	Record start date
End_Date	Record end date
Current_Flag	Active record indicator

Initial Data Load

First load from source.

SK	Customer_ID	City	Start_Date	End_Date	Current
1	101	Bangalore	2023	NULL	Y
2	102	Chennai	2023	NULL	Y

Scenario 1 – Type 1 Change

Customer updates email.

Before update

Customer_ID Email

101	rohith@gmail.com
-----	------------------

After update

Customer_ID Email

101	rohith123@gmail.com
-----	---------------------

Since this is **Type 1**, the value is simply overwritten.

Type 1 Implementation (Overwrite)

Steps:

1. Compare staging data with dimension table
2. Identify changed attributes
3. Update existing record

Example SQL

```
UPDATE customer_dim
SET email = s.email,
    phone = s.phone
FROM staging_customer s
WHERE customer_dim.customer_id = s.customer_id
AND customer_dim.current_flag = 'Y';
```

Result

SK Customer_ID Email

1	101	rohith123@gmail.com
---	-----	---------------------

Old value is **lost**.

Scenario 2 – Type 2 Change

Customer moves from **Bangalore** → **Hyderabad**.

Before change

	SK	Customer_ID	City	Start	End	Current
1	101		Bangalore	2023	NULL	Y

Business requirement:

Maintain **historical city changes**.

Type 2 Implementation Logic

When change is detected:

- 1 Update existing record
 - 2 Insert new record
-

Step 1 – Close Old Record

```
UPDATE customer_dim
SET end_date = CURRENT_DATE,
current_flag = 'N'
FROM staging_customer s
WHERE customer_dim.customer_id = s.customer_id
AND customer_dim.city <> s.city
AND customer_dim.current_flag = 'Y';
```

Result

	SK	Customer_ID	City	Start	End	Current
1	101		Bangalore	2023	2024	N

Step 2 – Insert New Record

```
INSERT INTO customer_dim
(
customer_sk,
customer_id,
name,
email,
city,
phone,
start_date,
end_date,
```

```

current_flag
)
SELECT
SEQ_CUSTOMER.NEXTVAL,
s.customer_id,
s.name,
s.email,
s.city,
s.phone,
CURRENT_DATE,
NULL,
'Y'
FROM staging_customer s;

```

Result

	SK	Customer_ID	City	Start	End	Current
1	101		Bangalore	2023	2024	N
3	101		Hyderabad	2024	NULL	Y

Now the **history is preserved**.

Combined Type1 + Type2 Implementation Using MERGE

Most modern pipelines (Snowflake / SQL DW) use **MERGE statements**.

Example:

```

MERGE INTO customer_dim t
USING staging_customer s
ON t.customer_id = s.customer_id
AND t.current_flag = 'Y'

```

```

WHEN MATCHED AND
(
t.email <> s.email
OR t.phone <> s.phone
)

```

```

THEN UPDATE SET
t.email = s.email,
t.phone = s.phone

```

```

WHEN MATCHED AND
t.city <> s.city
THEN UPDATE SET
t.end_date = CURRENT_DATE,
t.current_flag = 'N'

```

WHEN NOT MATCHED THEN

INSERT

(

customer_sk,

customer_id,

name,

email,

city,

phone,

start_date,

end_date,

current_flag

)

VALUES

(

SEQ_CUSTOMER.NEXTVAL,

s.customer_id,

s.name,

s.email,

s.city,

s.phone,

CURRENT_DATE,

NULL,

'Y'

);

Real-Time ETL Workflow

Daily pipeline process:

- 1 Load source data into **staging table**
 - 2 Compare staging with **dimension table**
 - 3 Identify changes
 - 4 Apply **Type1 or Type2 logic**
 - 5 Update dimension table
-

Real Production Example

Industry: **Banking**

Customer Address Change

Old record

Customer Address	Start	End
------------------	-------	-----

101	Bangalore	2022	NULL
-----	-----------	------	------

Customer moves to Mumbai

New table state

Customer Address	Start	End
------------------	-------	-----

101	Bangalore	2022	2024
-----	-----------	------	------

101	Mumbai	2024	NULL
-----	--------	------	------

Now analysts can answer:

- Customer distribution by city
 - Customer migration trends
 - Regional sales analysis
-

Performance Best Practices

When implementing SCD pipelines:

- Use **surrogate keys**
 - Use **MERGE statements**
 - Index business keys
 - Load data through **staging tables**
-

Common Interview Questions

Which SCD type overwrites data?

Type 1

Which SCD type maintains full history?

Type 2

What columns are required for Type 2?

Common columns:

- Start_Date
 - End_Date
 - Current_Flag
 - Version
-

Why use Surrogate Key in SCD?

Because **business keys can repeat when history records are created.**

Quick Summary

In real data warehouse projects:

SCD Type 1

Used when history is not required.

SCD Type 2

Used when business requires full historical tracking of dimension changes.

Most production systems use a **combination of Type1 and Type2** to balance performance and historical analysis.